



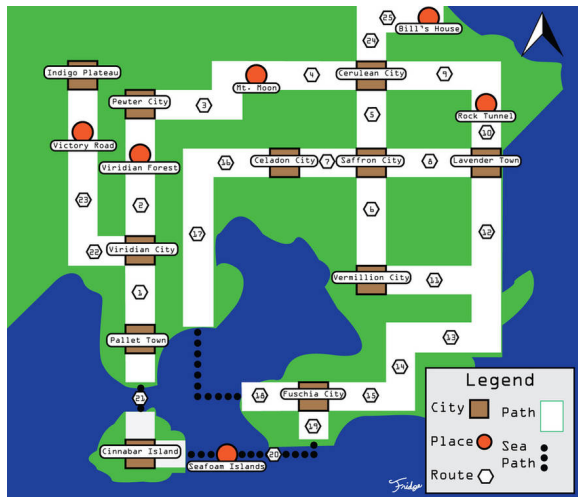
Taller 6

Objetivos

- Implementar grafos en C utilizando matrices y listas de adyacencia.
- Utilizar arreglos de visitados para recorrer grafos mediante algoritmos como DFS, evitando ciclos infinitos.

Introducción

En la región de Kanto, los entrenadores Pokémon deben viajar constantemente entre ciudades, rutas, cuevas y gimnasios. Algunos caminos conectan directamente dos lugares, mientras que otros obligan a pasar por rutas intermedias.



El Profesor Oak quiere mejorar la Pokédex para que esta no solo registre Pokémon, sino que también pueda representar el mapa de la región como un grafo. Cada ciudad o ruta será representada como un nodo, y cada camino entre dos lugares será representado como una arista.

De esta forma, la Pokédex podrá responder preguntas como: ¿existe un camino entre Ciudad Paleta y Ciudad Celeste?, ¿qué lugares se visitan al recorrer el mapa?, o ¿existen ciclos en el mapa que permitan volver a un mismo lugar por caminos distintos?

¡Viajando por el Mundo Pokemon!



Problema

Para poder viajar por el mundo Pokemon, diremos que cada lugar del mapa será considerado un nodo, y cada camino entre dos lugares será considerado una arista.

Tu misión será implementar esta estructura en C utilizando dos representaciones distintas: una matriz de adyacencia y una lista de adyacencia.

Además, deberás implementar algoritmos de recorrido sobre el grafo utilizando un arreglo de visitados, y finalmente determinar si el mapa contiene ciclos.

Entidades

Lugar: Cada ciudad o punto importante del mapa Pokémon será representado por un número entero entre 0 y $N - 1$.

Camino: Una conexión entre dos lugares del mapa. Como los entrenadores pueden recorrer los caminos en ambos sentidos, el grafo será no dirigido.

Mapa: Corresponde al grafo que almacena todos los lugares y caminos de la región. Este grafo deberá representarse tanto mediante una matriz de adyacencia como mediante una lista de adyacencia.

Flujo

Primero recibirás un entero K , que indica la cantidad de eventos a procesar. Luego recibirás K eventos, cada uno correspondiente a una operación sobre el mapa Pokémon.

Los eventos deberán procesarse en el orden en que aparecen y siempre comenzara con `CREATE-MAP {N}`

Inicialmente, el mapa estará vacío. A medida que se procesen los eventos, se irán agregando caminos entre ellos, recorrer el grafo y determinar si existen ciclos dentro del mapa.

Parte Formativa - Eventos

CREATE-MAP {N} En este evento se crea un mapa con N lugares, numerados desde 0 hasta $N - 1$. Tu tarea será inicializar la matriz de adyacencia del mapa.

El resultado esperado debe mostrar el siguiente output:

output:	
1	Mapa con {N} lugares creado.

ADD-ROUTE {A} {B} En este evento se agrega un camino entre los lugares A y B . Los valores de A y B siempre serán valores de 0 a $N - 1$. Como el grafo es no dirigido, se debe agregar la conexión en ambos sentidos.

Para la matriz de adyacencia se debe registrar el camino de A a B actualizando sus valores de 0 a 1, además de registrar el camino de B a A también, dado que es no dirigido:

El resultado esperado debe mostrar el siguiente output:

output:	
1	Camino actualizado entre {A} y {B}.

PRINT-MATRIX En este evento se debe imprimir la matriz de adyacencia actual del grafo. Con este evento, podrán ver el estado actual y los caminos presentes de su mapa.

El resultado esperado debe mostrar el siguiente output:

output:	
1	Matriz de adyacencia:
2	0 0 1 0 1
3	0 0 1 0 0
4	1 1 0 1 0
5	0 0 1 0 1
6	1 0 0 1 0

CAN-TRAVEL {A} {B} En este evento deberás determinar si es posible viajar desde el lugar A hasta el lugar B utilizando únicamente las rutas que han sido registradas previamente en el mapa Pokémon.

Es importante notar que no es necesario que exista una ruta directa entre A y B . Basta con que exista algún camino formado por una o más rutas intermedias.

Para resolver este evento, deberás implementar una búsqueda sobre el grafo utilizando un arreglo de visitados. Se puede ocupar DFS o BFS, por simplicidad, utilizaremos el enfoque de DFS visto anteriormente en el taller de Backtracking.

En ambos casos, el arreglo `visited` permitirá evitar visitar repetidamente los mismos lugares y prevenir ciclos infinitos.

Tu tarea será implementar en C una función similar a la siguiente:

```
1 bool can_travel(Graph* graph, int current, int target, bool visited[]);
```

El algoritmo debe seguir los siguientes pasos:

1. Si el lugar actual `current` es igual al lugar destino `target`, retornar `true`.
2. Marcar el lugar actual como visitado.
3. Recorrer todos los vecinos del lugar actual.
4. Para cada vecino que no haya sido visitado, llamar recursivamente a `can_travel`.

5. Si alguno de los llamados recursivos retorna **true**, entonces retornar **true**.

6. Si ningún vecino permite llegar al destino, retornar **false**.

El resultado esperado debe mostrar el siguiente output:

output:
1 Existe un camino entre {A} y {B}.

output:
1 No existe un camino entre {A} y {B}.

PROPUESTO: Una vez implementado **CAN-TRAVEL** con DFS, ahora intente llegar al mismo resultado, pero esta vez utilizando BFS visto en clases.

Parte Evaluada - Grupo 2

En esta parte del taller trabajarás con consultas sobre un grafo no dirigido ya construido.

a) Implementacion de COUNT-ZONES

En este evento deberás determinar cuántas zonas conectadas existen actualmente en el mapa.

Diremos que una **zona conectada** corresponde a un conjunto de lugares donde es posible viajar entre cualquier par de lugares utilizando las rutas registradas en el mapa. Si existen lugares que no pueden alcanzarse entre sí, entonces pertenecen a zonas distintas.

Para resolver este evento, deberás recorrer el grafo utilizando DFS y un arreglo de visitados. La idea es revisar todos los lugares del mapa y, cada vez que se encuentre un lugar que aún no ha sido visitado, iniciar un DFS desde ese lugar. Todos los lugares alcanzados por ese DFS pertenecen a la misma zona conectada.

Para esta parte, ya cuentas con la estructuras implementadas en la parte formativa. Se pide que implementes las siguientes funciones para la realización de este ítem:

funciones a implementar:

```
1 | int count_zones(Graph *graph);  
2 | void dfs_zone(Graph *graph, int current, bool visited[]);
```

La función **dfs_zone** debe recorrer todos los lugares alcanzables desde el nodo **current**. El algoritmo debe seguir los siguientes pasos:

1. Marcar el lugar actual **current** como visitado.
2. Obtener la lista de vecinos del nodo actual usando la lista de adyacencia del grafo.
3. Recorrer todos los vecinos del nodo actual.
4. Si un vecino no ha sido visitado, llamar recursivamente a **dfs_zone** desde ese vecino.
5. Terminar cuando no queden vecinos sin visitar desde el nodo actual.

La función **count_zones** debe contar cuántas zonas conectadas existen en el mapa. El algoritmo debe seguir los siguientes pasos:

1. Crear un arreglo **visited** de tamaño **graph->n**.
2. Inicializar todos los valores de **visited** como **false**.
3. Inicializar un contador **zones** en 0.
4. Recorrer todos los lugares del mapa desde 0 hasta **graph->n - 1**.
5. Si el lugar actual no ha sido visitado:
 - Aumentar **zones** en 1.
 - Llamar a **dfs_zone** desde ese lugar.
6. Al finalizar el recorrido, liberar la memoria utilizada por **visited**, si corresponde.
7. Retornar el valor de **zones**.

Finalmente, al procesar el evento **COUNT-ZONES**, se debe llamar a la función **count_zones** e imprimir la cantidad de zonas conectadas encontradas. El resultado esperado debe mostrar el siguiente output:

output

```
1 | El mapa tiene {Z} zonas conectadas.
```

Donde **Z** corresponde a la cantidad de zonas conectadas presentes actualmente en el mapa.

b) Ejercicio Teórico DISCONNECTED-PAIRS

Considere un grafo no dirigido $G = (V, E)$ con N nodos numerados desde 0 hasta $N - 1$. Diremos que dos nodos u y v están **conectados** si existe un camino entre ellos. En caso contrario, diremos que el par (u, v) está **desconectado**.

Su objetivo será diseñar un algoritmo que calcule cuántos pares de nodos distintos no pueden alcanzarse entre sí. Para resolver este problema, puede utilizar como base la idea del evento COUNT-ZONES. Sin embargo, ahora no basta con contar cuántas componentes conectadas existen, sino que también será necesario conocer el tamaño de cada componente.

Ejemplo Considere el siguiente grafo no dirigido representado mediante una matriz de adyacencia:

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Este grafo tiene las siguientes componentes conectadas:

$$\{0, 1, 2\}, \{3, 4\}, \{5\}$$

Los tamaños de los componentes son 3, 2 y 1.

Los pares desconectados se forman entre nodos que pertenecen a componentes distintas. En este caso, la cantidad de pares desconectados es 11, ya que existen 11 pares de nodos que no pueden alcanzarse entre sí. $\{0, 3\}, \{0, 4\}, \{0, 5\}, \{1, 3\}, \{1, 4\}, \{1, 5\}, \{2, 3\}, \{2, 4\}, \{2, 5\}, \{3, 5\}$ y $\{4, 5\}$.

- a) Explique cómo se puede usar y modificar DFS para encontrar el tamaño de cada componente conectada del grafo.
- b) Escriba el pseudocódigo de un algoritmo que calcule la cantidad de pares de nodos desconectados. puedes utilizar una funcion auxiliar llamado `int dfs_component_size(Graph *graph, int current, bool visited[]);` y una funcion principal llamado `long long count_disconnected_pairs(Graph *graph);`
- c) Explique cuál es la complejidad temporal de su algoritmo en términos de V y E . justificando cuántas veces se visitan los nodos y cuántas veces se revisan las aristas.

BONUS

Dada la solución propuesta para el evento DISCONNECTED-PAIRS, impleméntela en C. En este caso, tendrá que completar las funciones entregadas:

```
int dfs_component_size(Graph *graph, int current, bool visited[]);
long long count_disconnected_pairs(Graph *graph);
```

El resultado esperado debe mostrar el siguiente output:

output:

```
1| Existen {N} pares desconectados en el mapa actual.
```

Input

Importante: En este taller trabajarás sobre un grafo, por lo que el flujo del programa estará basado en operaciones sobre esta estructura.

El archivo de input comenzará con un número **K** que corresponde a la cantidad de eventos a recibir. Luego, se entregarán los **K eventos**, cada uno con la información necesaria para realizar una operación sobre el grafo.

Como ejemplo tenemos el siguiente input:

input

```
1 10 # Numero de eventos a recibir
2 CREATE-MAP 6
3 ADD-ROUTE 0 1
4 ADD-ROUTE 1 2
5 ADD-ROUTE 2 4
6 MIN-DIST 0 4
7 MIN-DIST 0 5
8 ADD-ROUTE 4 5
9 ADD-ROUTE 2 3
10 PRINT-MATRIX
11 VISIT-ALL 0
```

output

```
1 Mapa con 6 lugares creado.
2 Camino actualizado entre 0 y 1.
3 Camino actualizado entre 1 y 2.
4 Camino actualizado entre 2 y 4.
5 Distancia minima entre 0 y 4: 3.
6 No existe un camino entre 0 y 5.
7 Camino actualizado entre 4 y 5.
8 Camino actualizado entre 2 y 3.
9 Matriz de adyacencia:
10 0 1 0 0 0 0
11 1 0 1 0 0 0
12 0 1 0 1 1 0
13 0 0 1 0 0 0
14 0 0 1 0 0 1
15 0 0 0 0 1 0
16 Desde 0 se puede visitar todo el mapa.
```

Evaluación (Solo parte evaluada)

La nota de tu taller es calculada a partir de testcases privados de input/output que se evaluarán cada uno en forma binaria, es decir, 1 punto por output correcto y 0 puntos por output incorrecto.

Se entregarán testcase publicos para que puedas probar tu implementación durante el taller.

Ademas para la parte teorica del evaluado, se espera una respuesta en sus propias palabras que se evaluara de manera manual. Por lo que se espera que el estudiante pueda explicar con sus propias palabras ya sea en pseudocodigo o en un parrafo como resolveria el problema.

Finalmente el Bonus tambien se evaluara con test cases.

Integridad académica e IA

Cualquier acto deshonesto o fraude académico esta estrictamente prohibido. Se prohíbe el uso de herramientas de inteligencia artificial y el uso o manipulación de otros dispositivos electrónicos como laptops, tablet, teléfonos durante la evaluación.

Proceso de Entrega

Conexión a internet

Una vez iniciado sesión en el escritorio, seleccionar el icono de redes arriba a la derecha. "*availablenetwork*" → "*eduroam*". Para conectarse a eduroam, deben tener la siguiente configuración:

- Tener seleccionado la opción "No CA certificate is required".
- En la opción de "*Inner authentication*" seleccionar "*PAP*".
- Luego debe ingresar tus credenciales usuario uc. **IMPORTANTE:** en el usuario debes incluir el @, no solo el nombre.
- Ejemplos correctos: "*nombre@uc.cl*"; "*nombre@estudiante.uc.cl*", Incorrecto: "nombre" sin el @.

Descargar taller

Para descargar el código base que con el que trabajaran, deben abrir una terminal, estar conectados a internet y ejecutar `descargar-taller nombre_archivo.zip`

Donde el *nombre_archivo* debe ser reemplazado con el nombre que se avisara al inicio de cada taller.

Editor de código

El editor de código de la imagen se llama "*lite - xl*". Para abrir su trabajo tienen varias alternativas, las recomendables son:

- Abrir el editor de código, buscar la sección de "*file*" y seleccionar "*Open*" y buscar la carpeta donde quieres trabajar.
- Ir al explorador de archivos, buscar la carpeta donde quieres trabajar, hacer click derecho en la carpeta y seleccionar "*Openwith...*" y seleccionar el editor.
- **IMPORTANTE:** El editor no tiene autoguardado, por lo que cada vez que hagan cambios deben usar "*CTRL + S*" para guardar. Si ustedes no guardan sus archivos e intentan compilar, el proceso de compilado no guarda automáticamente, por lo que su código se compilara con la ultima versión que hayan guardado, **DEBEN GUARDAR ANTES DE COMPILAR.**

Espacio de trabajo

Al abrir el explorador de archivos o una terminal, verán una carpeta llamada código. Es **FUNDAMENTAL** que todo el trabajo que hagan lo tengan dentro de esa carpeta. Eso les asegura que en caso de que la imagen falle y tengan que reiniciar el computador, todo lo que está guardado en esa carpeta se mantendrá.

Uso del entorno de trabajo

En su carpeta de trabajo tendrán al menos los siguientes elementos:

- src: carpeta con su código
- makefile: archivo para compilar
- tests: carpetas de tests publicas
- run.sh: script para ejecutar todos los tests

Para compilar, en una terminal usen:

```
make
```

Para usar el script que ejecuta todos los tests, usan:

```
./run.sh
```

El run.sh genera una carpeta llamada outputs que guarda para cada test, el output de su programa. **Si desean ejecutar un único test** ocupan

```
./ejecutable ruta_al_input_del_test.txt archivo_output.txt
```

Donde ejecutable es el nombre del archivo que se genera al compilar, puedes revisarlo en las carpetas dentro de src, ejemplo de ejecutables: "pokemones", "magnemites". Ejemplo de ruta al archivo de input: tests/F1/pokemones/easy-0/input.txt y output.txt es un archivo txt que crean y es donde ira su salida.

En caso de querer debugear algún error de un test en específico usar **valgrind**

```
valgrind ./ejecutable ruta_a_input.txt output.txt
```

Entrega Taller

Para entregar, también es importante que estén conectados a internet, entrar en una terminal, posicionarse en la ruta donde esta la carpeta de su entrega (no dentro de la carpeta misma). Finalmente, ejecutar

```
entregar-taller nombre_carpeta TX nAlumno1_nAlumno2_...
```

Donde: "*nombre_carpeta*" es el nombre de la carpeta que quieren entregar, por ejemplo: "*T2_evaluado_G2*". "*TX*" donde X corresponde al taller que se esta entregando: por ejemplo: "T2". Y *nAlumno1_nAlumno2_...* que son los números de alumnos de los integrantes del grupo que trabajaron separados por "_" (Si están realizando este taller de manera solitaria, solamente escriben su numero de alumno).
Ejemplo Correcto:

```
12345678_23456789 20387645 20387645_12345678_23456789
```

Ejemplo Incorrecto:

```
1234567823456789 Nombre1_12345678 Nombre1_Nombre2_Nombre3
```